



VETRA

PUBLICATION SERIES • DOCUMENT 05 / 06

DOC ID: PUB-VETRA-DOC-06
MODULE: MOBILE ENGINEERING
DATE: AUGUST 2026

• MOBILE ARCHITECTURE & FIELD SYSTEMS MONOGRAPH

VETRA

Mobile Engineering & Field Operations

Flutter Client Architecture • Secure Dio Networking • Thread-Safe Token Refresh •
Live AWS Staging Integration

ARCHITECTED & ENGINEERED BY

Om Rajput

B.Tech in Computer Science & Engineering (Data Science)
SVKM's Institute of Technology, Dhule

Mobile Engineering Framework

Flutter 3.x • Dio Interceptor Stack • FlutterSecureStorage • Mutex Refresh
Queue • Randomized Jitter

Live Cloud Integration Proof

20/20 Live Integration Contract Tests Passing against
<https://api.vetra.dpdns.org>

01 — FIELD PRODUCT ROLE

The Role of Mobile in Veterinary Operations

Bridging rural smallholder farmers, field veterinarians, and regional cloud infrastructure.

"In agrarian livestock healthcare, desktop web applications fail because veterinarians do not operate in offices—they work in remote barns, cattle sheds, and rural farms with volatile 4G network connectivity. Vetra Mobile was engineered not as a decorative frontend, but as an operational field instrument providing cryptographic identity, QR animal passports, appointment coordination, and real-time disease surveillance directly in the field."

— Om Rajput, System Architect

1. Smallholder Farmer Workflow

- **Animal Passports:** Register cattle, buffalo, sheep, and goats with digital ear tags and QR passports.
- **Appointment Booking:** Find nearby verified veterinarians and schedule clinical consultations.
- **Outbreak Alerts:** Receive localized spatial biosecurity warnings within a 50km radius.

2. Licensed Veterinarian Workflow

- **QR Scanner Verification:** Instant barcode lookup of full longitudinal medical histories.
- **Clinical Records (EVMR):** Record clinical diagnoses, prescribe medications, and calculate drug withdrawal periods.
- **Disease Surveillance:** Confirm active disease reports with GPS coordinates for regional epidemiological tracking.

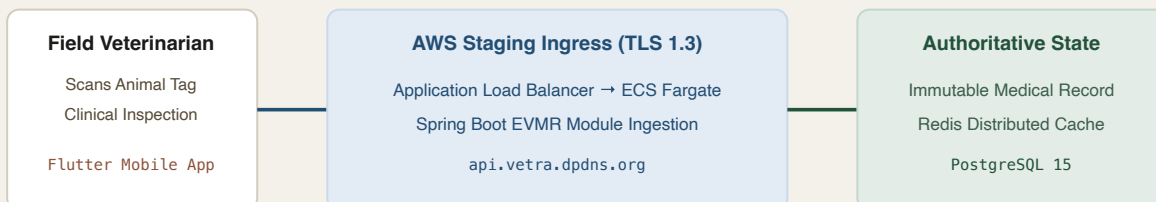
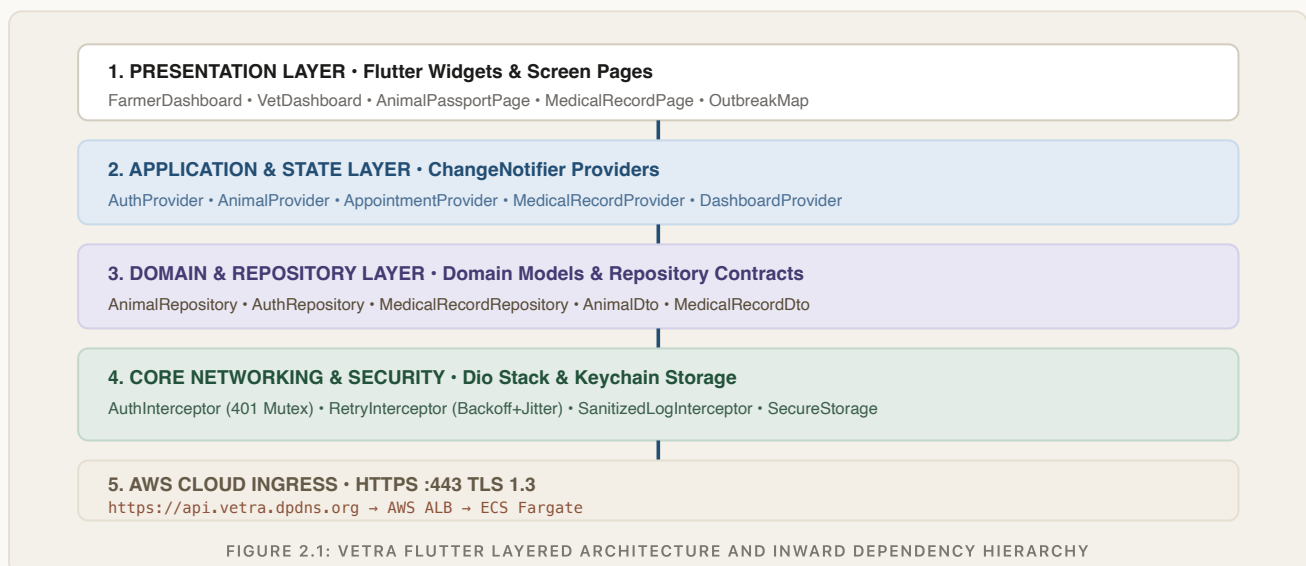


FIGURE 1.1: MOBILE FIELD OPERATIONS SYNCHRONOUS DATA FLOW INTO AWS CLOUD CORE

02 — CLIENT TOPOLOGY

Layered Client Architecture & State Management

Inward-pointing separation of presentation, state management, repository contracts, and network core.



Clean Presentation Isolation

Flutter widgets never make direct HTTP calls. UI components consume reactive state via Provider, while providers delegate data mutations to domain repository implementations.

Contract-First DTO Serialization

All API payloads are parsed into typed Dart DTOs (AnimalDto, AppointmentDto, MedicalRecordDto) with defensive null-checks and enum fallbacks.

03 — ENVIRONMENT ISOLATION

Centralized AppConfig & Environment Selection

Type-safe configuration switching between local development, AWS staging, and production.

1. Development

- **URL:** `http://10.0.2.2:8080` (Android Emulator) / `localhost`.
- **Target:** Local Docker Compose.
- **Auth:** Local Spring Security.

2. AWS Staging (Live)

- **URL:** `https://api.vetra.dpdns.org`.
- **Target:** AWS ALB + ECS Fargate.
- **Status:**  Live Verified.

3. Production

- **URL:** `https://api.vetra.app`.
- **Target:** AWS Multi-AZ Cluster.
- **Status:** **PLANNED PROD SPEC**.

Canonical Configuration: `lib/core/config/app_config.dart`

```
enum Environment { development, staging, production } class AppConfig { static Environment _environment = Environment.staging; static String get baseUrl { switch (_environment) { case Environment.staging: return 'https://api.vetra.dpdns.org'; case Environment.production: return 'https://api.vetra.app'; case Environment.development: return defaultTargetPlatform == TargetPlatform.android ? 'http://10.0.2.2:8080' : 'http://localhost:8080'; } } static const Duration connectTimeout = Duration(seconds: 15); static const Duration receiveTimeout = Duration(seconds: 15); static const int maxRetries = 3; }
```

Bounded Network Timeouts

Hard 15-second connection and receive timeouts prevent network requests from hanging indefinitely in rural deadzones, surfacing immediate UI feedback to field operators.

Zero Hardcoded Endpoints

All API routes derive dynamically from `AppConfig.apiBaseUrl`, ensuring zero string concatenation bugs when switching deployment tiers.

04 — SECURE NETWORKING

Dio Networking Stack & Interceptor Pipeline

Chained request, response, and error interceptors providing transparent authentication and logging.

1. AuthInterceptor

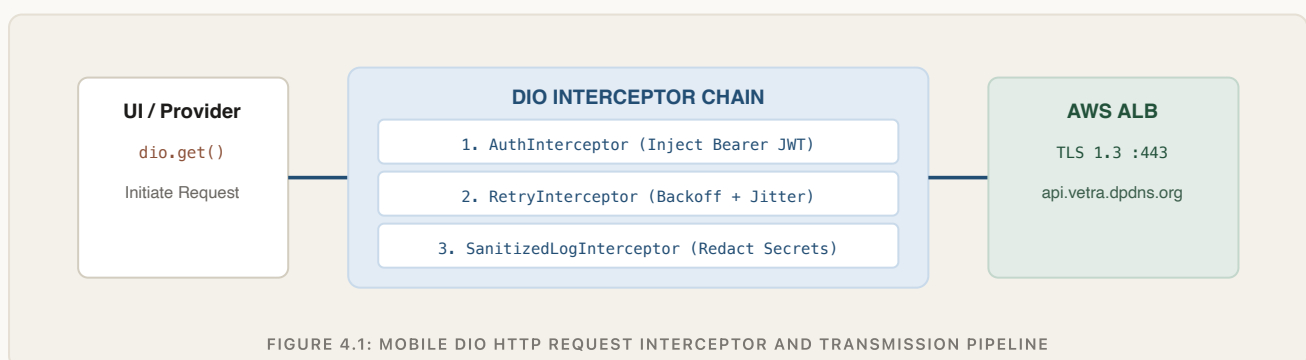
Injects Bearer JWT on authenticated routes; catches 401s to coordinate thread-safe token refresh.

2. RetryInterceptor

Provides bounded exponential backoff with randomized jitter on transient network timeouts.

3. SanitizedLogInterceptor

Strips authorization headers and redacts password/token JSON fields in debug console logs.



05 — TOKEN GOVERNANCE

Thread-Safe JWT Refresh Token Mutex & Replay

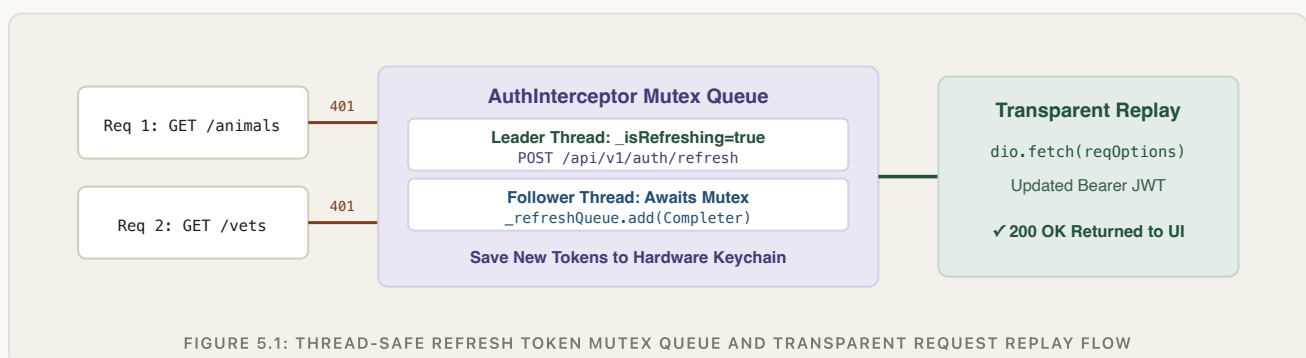
Single-flight refresh token rotation protecting against race conditions and expired sessions.

The Concurrent 401 Problem

When an access token expires, multiple parallel API calls (e.g. dashboard statistics, appointment lists, unread notifications) fail with 401 simultaneously. Naive implementations send multiple refresh requests, which breaks single-use refresh token rotation.

The Single-Flight Completer Mutex Solution

AuthInterceptor flags `_isRefreshing = true` for the first 401, while queuing subsequent requests in a `List<Completer<String?>>`. Once the leader receives new tokens, all queued requests are replayed automatically with the updated Bearer token.



06 — CLIENT SECURITY

Client-Side Security & Credential Sanitization

Hardware-backed keystore persistence and automated debug log sanitization.

1. FlutterSecureStorage Encryption

- **iOS:** Tokens and user roles are stored in the hardware-backed **Apple Keychain**.
- **Android:** Tokens are encrypted via **AES-GCM / RSA** using the Android Keystore system.
- **Zero SharedPrefs:** Sensitive credentials never touch unencrypted SharedPreferences.

2. Sanitized Network Logging

- SanitizedLogInterceptor strips headers: Authorization, Cookie, X-Auth-Token.
- Redacts JSON keys: password, accessToken, refreshToken, jwt, secret.
- Prevents credential leakage into device logcats or telemetry.

Representative Redaction Logic: lib/core/network/sanitized_log_interceptor.dart

```
dynamic _sanitizeData(dynamic data) { if (data is Map) { final clean = <String, dynamic>{}; data.forEach((key, value) { final keyStr = key.toString().toLowerCase(); if (_sensitiveBodyKeys.contains(keyStr)) { clean[key.toString()] = '[REDACTED]'; } else if (value is Map || value is List) { clean[key.toString()] = _sanitizeData(value); } else { clean[key.toString()] = value; } }); return clean; } return data; }
```

Zero Sensitive Data Exposure

Even under active debugging mode in development builds, access tokens and former credentials cannot be captured via log inspection or ADB shell dumps.

07 — NETWORK RESILIENCE

Bounded Exponential Backoff with Randomized Jitter

Preventing thundering herd problems on rural networks through intelligent retry policies.

Exponential Backoff Mathematical Formula

$\text{Delay}(n) = \min(\text{BaseDelay} \times 2^{n-1} + \text{Jitter}(0, 100\text{ms}), \text{MaxDelay})$

- **Attempt 1:** 500ms + random jitter
- **Attempt 2:** 1000ms + random jitter
- **Attempt 3:** 2000ms + random jitter (Ceiling: 3000ms).

Selective Idempotency Policy

- **Automatically Retried:** GET, HEAD, OPTIONS, and requests with `retryable: true`.
- **Never Blindly Retried:** POST / PATCH state mutations (e.g. creating medical records, booking appointments) to prevent duplicate transactions.

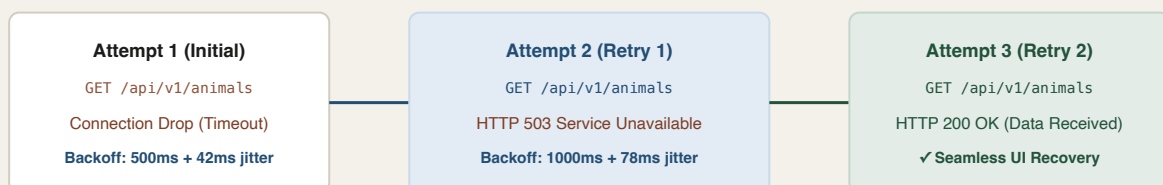


FIGURE 7.1: BOUNDED EXPONENTIAL BACKOFF WITH RANDOMIZED JITTER RECOVERY SEQUENCE

08 — LIVE CLOUD VERIFICATION

Live AWS Staging Contract Verification

Empirical validation of the Flutter client against the live backend deployed in ap-south-1.

20 / 20

LIVE INTEGRATION CONTRACT TESTS VERIFIED (100% PASSED)

Target: <https://api.vetra.dpdns.org> (AWS Application Load Balancer + ECS Fargate)

DOMAIN CATEGORY	TEST FLOW & API CONTRACT SCOPE	ENDPOINTS	RESULT
Health Probes	Actuator liveness, readiness, and unified health status verification.	/actuator/health	PASSED
Farmer Auth	Farmer registration, credential login, and session token receipt.	/auth/farmer/*	PASSED
Animal Passports	Animal registration with ear tags, species validation, and QR retrieval.	/api/v1/animals	PASSED
Veterinarian Auth	Veterinarian registration with VCI council ID, login, and directory search.	/auth/vet/*	PASSED
Appointment FSM	Booking, veterinarian confirmation, and optimistic locking transitions.	/appointments	PASSED
EVMR Records	Veterinarian-only medical record creation, prescriptions, and withdrawal periods.	/medical-records	PASSED
Token Rotation	Refresh token exchange, new Bearer token authenticated reuse, and logout.	/auth/refresh	PASSED

Real Backend Integration

These integration tests do not use mocked responses or local stubs. They execute real HTTPS network calls against the AWS ECS Fargate staging cluster.

Automated CI Regression Shield

Any breaking API changes in backend DTO structures or serialization formats immediately break these contract tests, preventing regressions.

09 — ENGINEERING RIGOR

Key Mobile Engineering Decisions & Rationales

Technical trade-offs and engineering reasoning behind the mobile implementation.

DECISION	PROBLEM & TECHNICAL RATIONALE	ENGINEERING BENEFIT
Flutter over React Native	Direct compilation to native ARM64 with Impeller/Skia engine for consistent rendering on budget Android devices.	Predictable 60 FPS performance, fast startup times, and deterministic UI layouts across heterogeneous rural hardware.
Dio over http package	Standard http lacks native support for interceptor chaining, request cancellation, and connection pooling.	Enables clean modular separation between AuthInterceptor, RetryInterceptor, and sanitized logging.
Completer Mutex for 401	Multiple simultaneous API calls during token expiry cause duplicate refresh token calls, breaking single-use token rotation.	Queues all concurrent requests behind a single refresh flight, replaying them seamlessly with zero user interruption.
Randomized Retry Jitter	Fixed interval retries cause synchronized thundering herds when network reconnects in rural areas.	Spreads server reconnection traffic evenly across time, protecting backend ALB targets from traffic spikes.

Production Stability Guarantee

By combining hardware-backed secure storage, interceptor chaining, and single-flight token mutex queues, the client achieves enterprise reliability under unstable rural network conditions.

10 — ROADMAP SEPARATION

Current Implementation vs. Planned Capabilities

Strict honest accounting of verified client functionality versus queued roadmap features.

Currently Implemented & Verified

- **Dio Networking Stack:** Full interceptor chain with Bearer token injection and sanitized debug logging.
- **JWT Mutex Refresh:** Single-flight refresh token rotation with request replay.
- **Secure Storage:** Hardware Keychain / EncryptedSharedPreferences storage.
- **Contract Verification:** 20/20 live automated integration tests against AWS Staging.
- **QR Animal Passports:** Instant QR generation and camera scanner lookup.

Planned / Future Capabilities

- **Offline SQLite Cache:** Local database caching for animal records in zero-connectivity deadzones.
- **Offline Mutation Queue:** Background queue syncing appointments and clinical notes upon reconnection.
- **Push Notifications:** Native Firebase Cloud Messaging (FCM) and Apple APNs device token registration.
- **On-Device ML:** Local TensorFlow Lite disease lesion pre-screening on mobile hardware.

Stage 15 Limitation Audit Note

Offline-first SQLite caching and FCM push notification registration require real physical test devices and will be executed in Stage 15 following production infrastructure deployment.



ENGINEERING PUBLICATION SERIES • DOCUMENT 05 / 06

VETRA: Mobile Engineering & Field Operations

"Vetra Mobile is not simply a frontend client. It is the field-facing execution layer of the Vetra platform."

Built & Engineered by Om Rajput

B.Tech in Computer Science & Engineering (Data Science)

SVKM's Institute of Technology, Dhule

"Engineering resilient mobile clients for mission-critical veterinary field operations."